双域协同 SM2 签名（2-of-2）实施规划 · A 与 B 分工

> 适用时间：2026-10-18（汇报后）至 2026-10-22（复赛提交） 使用方式：A 和 B 各自把"第 0 节 + 自己的任务卡"整段复制给执行 AI。每张任务卡都能单独交给 AI 完成，并写明了验收命令。 本文件所在目录的 参考实现/ 里有一份**已用 OpenSSL 双向验证通过**的协议参考代码，这是全部密码学正确性的"标准答案"。

---

0. 给所有执行 AI 的统一前言（每次派任务时都先贴这一节）

0.1 我们要做什么（一句话）

把现在的"2-of-3 FROST-Ed25519 门限批准"换成**双域协同 SM2 签名**：执行凭据的 SM2 私钥被拆成两份，一份在**授权域**（和授权服务器 AS 同一侧），一份在**执行域**（和网关同一侧）。授权域只有在**自己独立核验**委托链有效、未撤销、预算够用之后，才参与签名。两份合在一起产生的是一个**标准 SM2 签名**，网关用普通的 SM2 验签函数（铜锁 tongsuopy）就能验证。

0.2 协议数学（不要改，照抄参考实现）

记 SM2 曲线阶为 n，基点为 G，用户标识 ID 固定为 `1234567812345678`（与 GM-MVP-1 一致）。

- **密钥生成（不需要可信 dealer）**
  - 执行域 P1：随机取 $d1 \in [1, n-1]$，发送 $T1 = d1^{-1} \cdot G$
  - 授权域 P2：随机取 $d2 \in [1, n-1]$，计算公钥 $P = d2^{-1} \cdot T1 - G$（P 不能是无穷远点）
  - 此时 $(1+d)^{-1} = d1 \cdot d2 \bmod n$，完整私钥 d 从未在任何地方出现
  - 生成后必须做一次"确认签名"：对固定消息 `AGENT-GUARD-COSM2-KEYGEN-CONFIRM` 跑一次完整签名，并用标准验签通过
- **签名（消息 M）**
  1. 双方**各自**由结构化行动计算 M，再算 $e = SM3(Z_A \| M)$。**P2 绝不能接收 P1 算好的 e 或 M 的摘要**，必须拿到结构化行动后自己重算，这是"授权域控制签什么"的关键。
  2. P1：随机取 k1，发送 $Q1 = k1 \cdot G$
  3. P2：先校验 Q1 在曲线上且不是无穷远点；随机取 k2、k3；计算 $(x1, \cdot) = k3 \cdot Q1 + k2 \cdot G$，$r = (e + x1) \bmod n$，若 $r = 0$ 则重取；再算 $s2 = d2 \cdot k3 \bmod n$，$s3 = d2 \cdot (r + k2) \bmod n$；发送 (r, s2, s3)，**立即销毁 k2、k3**
  4. P1：计算 $s = d1 \cdot k1 \cdot s2 + d1 \cdot s3 - r \bmod n$；若 $s = 0$ 或 $s = n - r$ 则整轮重来；用公钥 P 做标准 SM2 验签，**通过才输出 (r, s)**；无论成败都销毁 k1
- **正确性**：$k = k1 \cdot k3 + k2$，$s = d1 \cdot d2 \cdot (k + r) - r = (1+d)^{-1}(k + r) - r$，就是标准 SM2 签名。
- **安全模型**：半诚实。恶意安全（零知识证明）不在本轮范围内，文档里必须如实写明。

0.3 硬性规则（违反任何一条，交付即作废）

1. **不改**迁移 001–009，不改 V1/V2/V3 门限消息的字节格式，不改 GM-MVP-1 已有的签名语义。新东西一律"新增版本或新增 profile"。
2. **网关验签只用 `agent_guard.crypto.sm.verify_sm2_message`（铜锁）**，不准用我们自己写的曲线代码验签。自写曲线运算只用于产生份额。
3. 不准用计时器、硬编码或 mock 伪造"成功"；不准为了让测试变绿删掉负例。
4. 日志、结果 JSON、前端里**不出现** d1、d2、k1、k2、k3、完整令牌、私钥文件内容。

5. 份额文件权限为 0600，所在目录权限为 0700；执行域进程只读 `exec/` 目录，授权域进程只读 `authz/` 目录。
6. k1 只能存在内存中，一次签名会话用完即销毁；P2 不保存任何跨轮状态。
7. 每次交付都附上：改了哪些文件、实际运行的命令和原样输出、已知限制。不准只说"做完了"。
8. 在分支 `codex/cosm2-2p-<A或B>` 上工作，不直接改 main。由 B 担任唯一集成者。

0.4 名词与常量（全组统一，不准各起各的名字）

名称	取值
方案名（配置）	`"sm2-2p"`（旧方案为 `"frost-ed25519"`）
批准 profile	`AG-COSM2-APPROVAL-1`
签名消息域前缀	`AGENT-GUARD-COSM2-ACTION-V1\n`（字节串）
行动字段	直接复用门限 V3 行动（V2 的 24 个字段加 `payment_digest`），即 `threshold/codec.py::parse_action_v3` 的输出顺序
签名格式	64 字节大端 `r‖s`，放进信封时用 base64url
公钥格式	SM2 SubjectPublicKeyInfo DER，配置中写 base64url
中文名	"双域协同 SM2 签名"，简称"双域共签"。不要叫"门限 SM2"，n=2 时应称"协同签名"

拒绝码（新增）：`AUTHZ_REVOKED`、`AUTHZ_EXPIRED`、`AUTHZ_PATH_MISMATCH`、`AUTHZ_SCOPE`、`AUTHZ_OVER_BUDGET`、`AUTHZ_POLICY_MISMATCH`、`EXEC_MISMATCH`、`COSIGN_ABORTED`。

---

1. 时间线与检查点

时间	B（协议与后端）	A（前端、独立核验、实验与图）	检查点
10-18 汇报后至晚上	B1 协同签名核心库；B2 份额存储与 keygen	A1 OpenSSL 独立核验脚本；A2 前端"双域共签"示意页骨架	—
10-19 上午	B3 授权域谓词门控签名者	A3 用参考实现交叉测试 B 的核心库	**G1 10-19 12:00**：核心库单测通过，公开向量被 OpenSSL 验过
10-19 下午至晚上	B4 网关按配置切换方案；B5 执行域签名者	A4 基准测试脚本	—
10-20 全天	B6 集成测试；B7 固定演示与桌面入口；B8（可选）x402 共签	A5 前端接入真实演示结果；A6 论文用图	**G2 10-20 18:00**：代码冻结

时间	B（协议与后端）	A（前端、独立核验、实验与图）	检查点
10-21	只修缺陷；配合 C 核对论文措辞	A7 跑基准并出图，截图录屏，全部交给 C	**G3 10-21 18:00**：数据和图冻结
10-22	打冻结标签，写启动说明	检查论文里的图和截图	提交

降级规则

- G1 未过：停止集成，论文改写为"协议设计 + 参考实现 + 公开验签向量"，B 转去把核心库和单测做扎实。
- G2 未过：网关集成不进论文"已实现"部分，只写独立原型；前端显示"独立原型，未接入采购"。
- B8（x402）和 B9（效果端验签）都是可选项，10-20 中午前 B6 没有全绿就都不做；只能做一个时，先做 B9。

---

2. B 的任务卡

B1 · 协同签名核心库（10-18 晚，约 3 小时）

目标：在 `src/agent_guard/cosign/` 新建纯计算模块，不涉及数据库和网络。

新增文件

- `src/agent_guard/cosign/__init__.py`
- `src/agent_guard/cosign/curve.py`：SM2 曲线参数与点运算（加法、倍点、标量乘、点在曲线上的校验）
- `src/agent_guard/cosign/protocol.py`：`ExecParty`（对应 P1）、`AuthzParty`（对应 P2），外加 `compute_e(public_point, message)`
- `src/agent_guard/cosign/encoding.py`：点与 SPKI DER 互转（可调用 `crypto/sm.py` 的 `serialize_sm2_public_key`；tongsuopy 支持 `ec.EllipticCurvePublicNumbers(x, y, ec.SM2()).public_key()`），r‖s 与 DER 互转（复用 `crypto/sm.py` 里的私有转换函数，或参照其实现）

步骤

1. 把 `参考实现/cosm2_reference.py` 的数学**逐行**搬过来，变量名可以改，公式一个字都不能改。
2. 随机数统一用 `secrets.randbelow(n - 1) + 1`。
3. `ExecParty.sign_round1()` 若已有未完成的会话，必须报错；`sign_round3()` 第一件事就是把 k1 取出并置空。
4. `AuthzParty.sign_round2(message: bytes, q1)` 内部自己算 e，函数签名里**不允许出现 e 参数**。
5. 最终验签调用 `verify_sm2_message(load_sm2_public_key(spki), message, der)`，也就是铜锁。
6. 在文件头注释写明：原型，标量乘不是常数时间，仅用于份额计算。

验收（新建 `tests/unit/test_cosm2_core.py`，要求不连数据库也能跑）

- 生成 50 组随机密钥并签名，铜锁 `verify_sm2_message` 全部通过
- 改消息 1 字节后验签失败；换一把公钥后验签失败

- P2 收到不在曲线上的 Q1、或无穷远点时拒绝
- P1 未开会话就调用 round3 时报错；round3 失败后再调用一次仍然报错（证明 k1 已销毁）
- 固定 d1、d2（测试专用常量）时，公钥与 `cosm2_reference.py` 计算结果一致
- 导出公开向量 `tests/cosm2-public-vector.json`，含 `public_key_spki_b64url`、`message_b64url`、`signature_rs_hex`，**不含任何私密值**

命令：`python -m pytest tests/unit/test_cosm2_core.py -q`（在现有 threshold runner 镜像里运行，因为它装了 tongsuopy）

B2 · 份额存储与密钥生成（10-18 晚，约 1.5 小时）

新增 `src/agent_guard/cosign/shares.py`、`tools/cosm2_keygen.py`

- 份额文件为 JSON：`{"scheme":"sm2-2p","role":"exec"|"authz","share_hex":"...","public_key_spki_b64url":"...","key_epoch":N}`
- `tools/cosm2_keygen.py DIR --key-epoch N` 会创建 `DIR/exec/share.json` 和 `DIR/authz/share.json`（权限 0600，目录 0700），以及 `DIR/public.json`（只含公钥和 epoch）
- keygen 过程模拟两方各自生成：先构造 `ExecParty` 得到 T1，再交给 `AuthzParty` 算出 P，最后跑确认签名
- 读取份额时检查权限是否过宽，过宽则拒绝；`role` 不匹配也拒绝

验收：单测覆盖权限过宽拒绝、role 不匹配拒绝、确认签名失败时不写入任何文件。

B3 · 授权域谓词门控签名者（10-19 上午，约 3 小时）★ 核心创新所在

新增 `src/agent_guard/cosign/authz_domain.py`

```
class AuthorizationDomainSigner:
    def __init__(self, *, share_dir, dsn_readonly, snapshot_provider,
policy): ...
    def respond(self, *, action: dict, leaf_access_token: str, q1_hex: str,
now: int) -> dict:
        """返回 {"r":..,"s2":..,"s3":..}，或抛出 CosignRefused(code=...)。"""
```

`respond` 必须**按顺序**完成以下检查，任何一项失败都抛出 `CosignRefused`，且不进行任何曲线运算：

1. `parse_action_v3(action)` 严格解析（复用 `threshold/codec.py`）
2. `audience`、`policy_version`、`key_epoch` 与本域配置一致，否则 `AUTHZ_POLICY_MISMATCH`；`expires_at` 合法且 `expires_at - now ≤ max_lifetime`，否则 `AUTHZ_EXPIRED`
3. 用 `PermissionSnapshotProvider.load(leaf_access_token, grant_id=action["grant_id"], now=now)` 验证从 root 到 leaf 的 AS 签名链（构造方式参照 `tests/fixtures/gateway.py::app_for`）。链上的 grant id 序列必须等于 `action["path_grant_ids"]`，holder 和 kid 必须一致，否则 `AUTHZ_PATH_MISMATCH`；`tool_id` 必须在令牌 scope 内，否则 `AUTHZ_SCOPE`
4. 用**只读连接**查询路径上每个节点（`ag_grants` 表的 `revoked`、`expires_at`、`amount_limit`、`amount_reserved`、`amount_settled`）：任一节点 revoked，则 `AUTHZ_REVOKED`；任一节点已过

期，则 AUTHZ_EXPIRED；若 action["amount_fen"] > min(amount_limit - amount_reserved - amount_settled)，则 AUTHZ_OVER_BUDGET

5. 全部通过后，message = COSM2_DOMAIN + 规范 JSON(action)，再调用 AuthzParty.sign_round2(message, q1)

注意

- 这一步**只读不写**，不预留预算。真正的预留仍在网关的接受事务里完成，所以"签名后被撤销"由网关拦截（已有机制）。
- 只读连接要单独建一个数据库角色，只授予 ag_grants 及签名快照相关表的 SELECT 权限。若时间不够，可以先复用连接串，但必须在 README 的局限里写明。
- 演示部署中两个域在同一台主机上，必须在局限里写明。

验收：单测或集成测试覆盖 5 种拒绝码各一次，外加通过一次；拒绝时确认没有调用 sign_round2（用计数器断言）。

B4 · 网关按服务端配置切换方案（10-19 下午，约 2.5 小时）

修改

- threshold/contracts.py：ThresholdApprovalPolicy 新增字段 scheme: str = "frost-ed25519"。当 scheme == "sm2-2p" 时，group_public_key 存放 SPKI DER 字节（不再是 32 字节）
- gateway/config.py::threshold_policy：threshold_approval 节允许多一个可选字段 "scheme"，取值只能是这两个；sm2-2p 的公钥写为 group_public_key_spki_b64url。缺少 scheme 时行为与现在**完全相同**
- threshold/codec.py：parse_envelope(envelope, *, profile=PROFILE) 增加 profile 参数；新增 cosm2_message(action) -> bytes（使用相同的规范 JSON 序列化、V3 字段顺序，前缀为 COSM2_DOMAIN）
- threshold/verifier.py::verify：按 self._policy.scheme 分支。sm2-2p 分支要求信封 profile 为 AG-COSM2-APPROVAL-1，签名是 64 字节，然后转 DER，调用 verify_sm2_message。其余比对（expectation、有效期、返回的 VerifiedThresholdApproval）**完全复用**

为什么这样改：VerifiedThresholdApproval 类型不变，binding.py 的接受事务绑定和 x402/accept.py 的支付绑定都能直接用，不必再改。

验收

- 原有门限单测和集成测试全部仍然通过（回归）
- 新单测：在 sm2-2p 配置下，FROST 信封被拒；在 FROST 配置下，COSM2 信封被拒；客户端无法通过请求头选择方案

B5 · 执行域签名者与协调流程（10-19 晚，约 2 小时）

新增 src/agent_guard/cosign/exec_domain.py、tests/fixtures/cosm2.py

- ExecutionDomainSigner.approve(bundle, action, authz_client) -> bytes（返回 ag-approval 头）流程如下：
    1. 核对 action 与请求事实一致：tool、idempotency_key、由请求体重算的 compute_params_digest、可信报价算出的金额等于 amount_fen，否则 EXEC_MISMATCH。

行动的构造方式参照 `tests/fixtures/threshold.py::approval_action`
2. `q1 = ExecParty.sign_round1()`
3. 调用 `authz_client.respond(action=..., leaf_access_token=..., q1_hex=...)`
4. `ExecParty.sign_round3(...)` 后组装信封：`{"profile":"AG-COSM2-APPROVAL-1","action":...,"signature":b64url(r‖s)}`，再调用 `encode_header`

- `authz_client` 先做进程内版本（P0）。若时间允许，再做子进程版本（P1）：`python -m agent_guard.cosign.authz_cli respond --share DIR/authz`，JSON 走标准输入输出，用来演示两域各自独立进程、只读自己的份额
- 测试夹具 `tests/fixtures/cosm2.py` 提供与 `RustThresholdSigner` 对等的接口：`enroll()`、`sign(action)`、`sign_refused(action)`、`header_for(action, sig)`

B6 · 集成测试（10-20 上午，约 3 小时）

新增 `tests/integration/test_cosm2_gateway.py`，使用真实 PostgreSQL、真实 AS 签发链和独立下游，写法参照 `tests/integration/test_threshold_gateway.py`。

编号	用例	期望
C1	双域正常共签	202，恰好一单；用同一批准重试仍是同一 operation_id
C2	不带批准	401 `APPROVAL_REQUIRED`
C3	**只有执行域**：只用 d1，s2/s3 随机伪造	401 `INVALID_SIGNATURE`，零订单
C4	伪造 64 字节签名	401 `INVALID_SIGNATURE`
C5	签名后篡改金额	409 `APPROVAL_MISMATCH`
C6	**先撤销再请求共签**	授权域抛出 `AUTHZ_REVOKED`，不产生签名，零订单
C7	先共签再撤销	网关 403 `REVOKED`，零新订单
C8	金额超出剩余预算	授权域 `AUTHZ_OVER_BUDGET`
C9	方案错配（FROST 部署收 COSM2 信封）	拒绝
C10	批准已过期	401 `EXPIRED`
C11	产出的签名用铜锁标准验签	通过（证明与普通 SM2 签名不可区分）

验收命令（沿用门限的 runner 镜像）：

```
docker run --rm -v "$PWD":/work -w /work -e PYTHONPATH=/work/src \
  -e AGENT_GUARD_TEST_DATABASE_URL agent-guard-threshold-runner:local \
  /opt/venv/bin/python -m pytest tests/unit/test_cosm2_core.py \
  tests/integration/test_cosm2_gateway.py
tests/integration/test_threshold_gateway.py -q
```

另外运行 `ruff check .` 和 `ruff format --check .`。

B7 · 固定演示与桌面入口（10-20 下午，约 2 小时）

- 新增 `tools/cosm2_gateway_demo.py`，照抄 `tools/threshold_gateway_demo.py` 的结构，用例换成 C1、C2、C3、C5、C6、C7。结果写到 `artifacts/cosm2-gateway-demo-live/<run_id>/result.json`
- result.json 字段与前端约定如下（A 依赖它，不准随意改）：


```
{"run_id":"...","passed":true,"completed_at":1760000000,"scheme":"sm2-2p",
 "public_key_fingerprint":"sm3 前 16 位 hex",
 "orders":1,"operations":1,"approvals":1,"resources_remaining":0,
 "cases":[{"name":"cosign-valid-one-
order","passed":true,"http_status":202},
          {"name":"missing-
approval","passed":true,"http_status":401,"code":"APPROVAL_REQUIRED"},
          {"name":"exec-domain-
alone","passed":true,"http_status":401,"code":"INVALID_SIGNATURE"},
          {"name":"tampered-
amount","passed":true,"http_status":409,"code":"APPROVAL_MISMATCH"},
          {"name":"revoked-before-
cosign","passed":true,"refused_by":"authz","code":"AUTHZ_REVOKED"},
          {"name":"revoked-after-
cosign","passed":true,"http_status":403,"code":"REVOKED"}],
 "timing_ms":
{"round1":0.0,"round2_with_checks":0.0,"round3_with_verify":0.0}}
```


- `tools/teacher_demo_server.py` 新增本机控制动作 `cosm2-run`（参照 `threshold-run` 的写法，公网仍只读）；`tools/threshold_desktop.py` 增加对应入口，或复制一份 `cosm2_desktop.py`
- 文档：新增 `docs/cosm2-gateway-20261020.md`，写清实现内容、复现命令、局限（同主机、半诚实、非常数时间、两域共用数据库只读角色）

B8（可选）· x402 付款意图由双域共签批准（10-20，仅在 B6 全绿时做）

因为 B4 让 `VerifiedThresholdApproval` 类型保持不变，`x402/accept.py::payment_binding` 不需要修改。要做的只有：

1. 在 `tools/x402_gateway_demo.py` 里加一个配置开关：网关用 `sm2-2p` 策略，批准改由 `tests/fixtures/cosm2.py` 产生，V3 行动中带 `payment_digest`
2. 跑通"正常付款只结算一次"和"篡改支付被拒"两例
3. 论文和文档里的准确说法是："付款意图必须经双域 SM2 共签批准；链上结算签名仍是 EIP-3009（secp256k1），未替换。"

B9（可选，优先级高于 B8）· 效果端独立验签

为什么要做：如果只有网关验签，那么"网关的验签代码本身被攻破"这种情况就挡不住。让真正产生效果的一方（模拟订单服务）也用公钥独立验一次共签签名，"即使网关被攻破也下不了单"这句话才成立。

- 网关转发给下游时附上原始 ag-approval 信封（下游接口新增一个可选字段，只在 sm2-2p 部署下要求必填）

- 下游 `src/agent_guard/tools/downstream.py`：用配置里的同一把公钥调用 `verify_sm2_message`，并核对信封里的 `amount_fen`、`items`、`idempotency_key` 与它收到的业务意图一致，不一致就拒绝下单
- 新增用例 C12："网关逻辑被绕过、直接调用下游但没有有效共签"，期望下游拒绝、零订单
- 做不完就不做，论文里把"效果端验签"写进后续工作，威胁模型里写明"假设网关验签逻辑可信"

---

3. A 的任务卡

A1 · OpenSSL 独立核验脚本（10-18 晚，约 1 小时）

新增 `tools/cosm2_vector_check.py`（A 独占）

- 读取 B1 产出的 `tests/cosm2-public-vector.json`，写出 PEM 公钥、消息和 DER 签名到临时目录，调用：`openssl pkeyutl -verify -pubin -inkey pub.pem -rawin -in msg.bin -sigfile sig.der -digest sm3 -pkeyopt distid:1234567812345678`
- 期望输出 `Signature Verified Successfully`；把消息改 1 字节后必须失败
- 写法直接参考 `参考实现/cosm2_xcheck_openssl.py`（已验证可用）
- **注意**：macOS 自带的 `openssl` 实际是 LibreSSL，不支持 SM2。请先 `brew install openssl@3`，再用 `$(brew --prefix openssl@3)/bin/openssl`，或者在 Docker/Linux 里运行
- 这一步就是论文里"产生的签名能被独立的标准国密实现验证"这句话的证据

A2 · 前端"双域共签"示意页（10-18 晚至 10-19，约 3 小时）

只改 `web/teacher-demo/` 的三个文件，遵守原有约束：CSP 为 `default-src 'self'`，不内联脚本和样式，不用 CDN；live、replay、illustrative 三种来源醒目区分。

1. 导航新增 `data-view="cosign"`，标签为"双域共签 · SM2"，放在"门限批准"之前；原"门限批准"按钮改名为"FROST 对照"
2. `app.js` 的 `SCENES` 中新增 `cosign`，结构照抄 `threshold`：
   - `kicker`："核心机制：双域协同 SM2 签名"
   - `title`："授权域点头，才能签出这一笔"
   - `lede`："执行凭据的私钥被拆成两份：一份在授权域，一份在执行域。授权域先自己查一遍委托链、撤销和预算，确认无误才交出它那一半；两半合成一个普通 SM2 签名，网关用标准国密验签就能验证。"
   - `lanes`：执行域（P1 · 网关侧）、授权域（P2 · AS 侧）、执行网关、授权与预算库、模拟订单
   - 示意事件顺序：形成行动 → 执行域发出 Q1 → 授权域核验委托链 ✓ → 核验撤销 ✓ → 核验预算 ✓ → 返回 (r, s2, s3) → 执行域合成并用标准 SM2 验签 ✓ → 请求带 ag-approval 到网关 → 接受，一单
   - 负例分支：撤销后再请求共签，**红叉挂在授权域节点**（这是真实的拒绝位置），显示 `AUTHZ_REVOKED`，网关和下游标为"未发生"
   - `delta` 对比三行：①普通单钥：网关被攻破就能任意签；②2-of-3 FROST（本项目旧版）：签名者不检查授权状态，非国密；③双域共签（本项目）：授权域独立核验后才出份额，产出标准 SM2 签名（标记为本项目）
   - `honest`："当前为两域协同（n=2），半诚实模型；两域部署在同一主机；向 t-of-n 的推广已有设计，尚未实现。"
3. 在 `EVENT_TYPES` 中新增 `cosign.round1`、`cosign.checked`、`cosign.refused`、`cosign.partial`、`cosign.combined`，并在 `narrate` 中写出中文旁白

4. 在 MODULES 中新增 exec-domain、authz-domain 两张说明卡，六项内容按现有格式填写
5. 总览页的"扩展能力与边界"改为："双域共签（SM2）：核心机制；FROST 2-of-3：对照实现；x402：本机测试链"

验收：node tests/test_teacher_demo_frontend_logic.cjs 通过；在 1366×768 下无重叠；示意页左上角来源标签显示"示意推演"。

A3 · 交叉测试 B 的核心库（10-19 上午，约 1 小时）

新增 tests/unit/test_cosm2_crosscheck.py（A 独占新文件）

- 把 参考实现/cosm2_reference.py 复制到 tests/fixtures/cosm2_reference.py
- 用例 1：固定 d1、d2，B 的 ExecParty/AuthzParty 算出的公钥，与参考实现逐坐标相等
- 用例 2：参考实现产生的签名，B 的验签路径（铜锁）能验过；B 产生的签名，参考实现的 verify 也能验过
- 用例 3：B 的 compute_e 与参考实现的 digest_e 在 20 条随机消息上完全一致
- 任何一项不过，立刻通知 B，并在 G1 检查点报告

A4 · 基准测试脚本（10-19 下午至晚上，约 2 小时）

新增 experiments/cosm2_bench/bench.py、experiments/cosm2_bench/plot.py（A 独占）

- 固定环境：threshold runner 镜像，记录 CPU 型号、Python 版本、tongsuopy 版本
- 预热 10 次，正式测 100 次，记录每次的原始耗时（单位 ms），写入 results/<日期>/raw.csv
- 测量项：
    1. 协同签名三个阶段 round1、round2（**不含**数据库谓词检查）、round3（含最终验签），以及总时间
    2. round2 含数据库谓词检查的总时间（需要 B3 完成后才能测；测不到就写"未测"）
    3. 单方 SM2 签名（铜锁 sign_sm2_message）
    4. SM2 验签（铜锁）
    5. FROST 2-of-3 sign-quorum（Rust 子进程，**注明包含进程启动开销**）
    6. 网关一次完整受理的端到端时间（从 B7 的 result.json 中取，样本少就如实标注样本数）
- plot.py 输出 fig-latency.pdf：箱线图或柱状图，标出中位数和 p95，用 matplotlib，不加装饰
- 输出 summary.json：每项的样本数 n、中位数、p95、最小值、最大值
- **禁止**挑选好看的结果、删除异常值，或混用不同机器的数据

A5 · 前端接入真实演示结果（10-20，约 2 小时）

- 在 FIXED_DEMOS 中新增 cosign，action: 'cosm2-run'；names 按 B7 的用例名翻译为中文（例如 exec-domain-alone→"只有执行域，签不出有效签名"，revoked-before-cosign→"已撤销，授权域拒绝出份额"）
- 收到真实结果后，左栏显示"本次现场运行"标签和每个用例的结果；没有结果时继续显示"示意推演"
- timing_ms 显示为"本次三轮耗时"，标注"单次运行，非统计结果"

A6 · 论文用图（10-20，约 3 小时，交给 C）

导出为矢量 PDF（用 draw.io、Figma 或 TikZ 均可），统一配色和中文字体，放在 复赛冲刺材料-20261018/论文模板/figures/：

1. `fig-system.pdf` 系统模型图：用户 → 三代理 → AS（授权域放在 AS 旁的虚线框内）→ 网关（执行域放在网关旁的虚线框内）→ 授权与预算库 → 下游与证据；标出**两个信任域的边界**，以及"智能体不持有任何份额"
2. `fig-protocol.pdf` 协议时序图：P1/P2 三轮消息，在 P2 处标出"谓词检查：委托链、撤销、预算"，最后标出"标准 SM2 验签"
3. `fig-negative.pdf` 负例矩阵：行是攻击（盗令牌、扩权、撤销后、篡改金额、单域签名、伪造签名、超预算），列是拒绝位置（AS / 授权域 / 网关验签 / 接受事务），格子里填拒绝码；数据**只能**来自真实运行
4. `fig-latency.pdf`：A4 产出

A7 · 截图、录屏与交付（10-21）

- 对总览、场景 1/3/6、双域共签（示意 + 真实结果）、真实记录页截图，要求原图、无标注，各留一份
- 录一段 3 分钟现场运行，画面里要能看到 run_id、日期和来源标签
- 把所有图、截图、`summary.json` 和 `raw.csv` 交给 C，并附一张表：哪张图对应哪个 run_id 或命令

---

4. A 和 B 之间的接口（先对齐，再开工）

接口	提供方 → 使用方	截止
`tests/cosm2-public-vector.json`	B1 → A1、A3	10-18 23:00
`ExecParty`、`AuthzParty`、`compute_e` 的函数签名	B1 → A3、A4	10-18 23:00
`result.json` 字段（见 B7）	B7 → A5	10-20 15:00（字段名今晚就定，内容后补）
拒绝码清单（见 0.4）	B → A2、A5、A6	已定
运行命令和 run_id 清单	B6、B7 → A6、A7、C	10-20 20:00

5. 交付给 C 的"可写进论文"清单（G2 时由 B 勾选）

- ☐ 两方协同 SM2 核心库及单测（附命令和输出）
- ☐ 公开向量已被 OpenSSL 3 独立验证（A1 输出）
- ☐ 授权域谓词门控，5 种拒绝码均有测试
- ☐ 网关按服务端配置切换方案，原门限回归全绿
- ☐ 集成用例 C1–C11 的通过数（**写实际数字**）
- ☐ 固定演示 run_id
- ☐ （可选）下游订单服务独立验共签签名（B9，决定威胁模型里能否写"网关被攻破也下不了单"）
- ☐ （可选）x402 付款意图由共签批准
- ☐ 局限清单：半诚实、n=2、同主机、非常数时间、只读角色是否真正隔离

未勾选的项目，论文中一律写成"设计"或"后续工作"。